University of Petroleum and Energy Studies

A Project Report on

Maze Runner: Algorithm Comparison

Submitted for partial fulfilment for award of the degree in

Masters of Computer Application (MCA)

SUBMITTED BY

Name: Sarthak Chauhan
SAP ID: 590030655
Batch – 1

Name: Aarti Somani
SAP ID: 590040559
Batch – 2

Name: Ravinder Singh
SAP ID: 590030651
Batch – 1

Name: Anshul Ghildiyal
SAP ID: 590031643
Batch – 1

Under the Supervision

Dr. Arnab Chatterjee
Faculty Supervisor

Batch - MCA 2026-28

Maze Runner: Algorithm Comparison Report

Abstract

Maze Runner is a native Android educational game developed to make path-finding algorithms visible and interactive. It generates a 10 × 10 maze with a fixed start cell at (0, 0) and a goal cell at (9, 9). A learner can watch two search strategies solve the same maze: a recursive depth-first search (DFS) that explores routes through backtracking, and a greedy best-first search that prioritizes cells using Manhattan distance to the goal. The application also includes a manual play mode so a user can navigate adjacent open cells and compare the number of moves with the algorithmic reference path.

The comparison interface records route length, explored nodes, execution time, the number of DFS paths retained for browsing, and whether the heuristic route matches the DFS shortest-path reference. The project demonstrates that speed, explored work and path quality are separate measures. The heuristic often focuses exploration, but its greedy scoring rule does not guarantee a shortest route. The report explains the actual application structure and code behavior; it does not introduce fabricated benchmark results. Measurements are produced dynamically when the algorithms are run on a selected maze.

Project particulars

Item	Details
Project title	Maze Runner: Algorithm Comparison
Institution	University of Petroleum and Energy Studies (UPES), Dehradun
Programme	Masters of Computer Application (MCA) - Artificial Intelligence and Machine Learning
Student	Sarthak Chauhan
Batch	MCA 2026-28
Platform / language	Native Android application / Java
Application version	1.0

Report contents

Section	Topic	Page
1	Introduction, problem statement and objectives	3
2	System overview and architecture	4
3	Maze representation and generation	5
4	Brute-force depth-first search	6
5	Greedy best-first heuristic search	7
6	Android implementation and interface	8
7	Performance evaluation and metrics	9
8	Testing and verification approach	10
9-10	Limitations and future enhancements	11
11-12	Learning outcomes and conclusion	12

1. Introduction

Path-finding is the task of identifying a route from a starting point to a destination while respecting the constraints of a search space. In a maze, each open cell can be treated as a node and a legal movement between neighboring cells as an edge. Search algorithms differ in the order in which they visit these nodes and in the information they use to decide what to explore next.

Maze Runner presents this idea as a small Android game. Instead of showing only a final route, it animates explored cells and selected paths. A comparison table then places the algorithm outputs side by side, allowing the learner to relate search behavior to measurable outcomes. Manual play adds a human route that can be compared with the reference path.

1.1 Problem statement

The project addresses the following educational problem: how can two different path-finding strategies be demonstrated on the same maze, and how can their differences in route length, amount of exploration and execution time be communicated clearly to a learner?

The application must generate a solvable grid, identify a route between fixed endpoints, display the search process, and report comparable measurements. The maze is deliberately small so that the exhaustive DFS search remains practical for interactive demonstration.

1.2 Objectives

- Create and reset a 10 × 10 maze with clearly identified start and goal cells.
- Implement recursive DFS with backtracking to explore routes and retain the shortest route found.
- Implement greedy best-first search using Manhattan distance and a priority queue.
- Animate explored cells and selected paths so that algorithm behavior can be observed.
- Report path length, explored nodes, execution time and route optimality in a common comparison table.
- Allow manual movement through adjacent open cells and compare the player route with the DFS reference.

1.3 Scope and assumptions

Area	Project scope
Grid	Fixed 10 rows × 10 columns.
Endpoints	Start at (0, 0); goal at (9, 9).
Movement	Four orthogonal directions; diagonal movement is not allowed.
Platform	Android application with a local, in-memory maze.
Search methods	Recursive DFS with backtracking; greedy best-first search.
Data storage	No database or server component is part of the supplied application.
Primary purpose	Algorithm learning and visual comparison, not a production navigation system.

2. System Overview and Architecture

Maze Runner is a single-module native Android application. The main activity manages screen state, user actions, maze generation, search execution and comparison values. A custom view is responsible for drawing the grid and its visual overlays. There is no separate backend service: the maze and algorithm calculations run locally in the application.

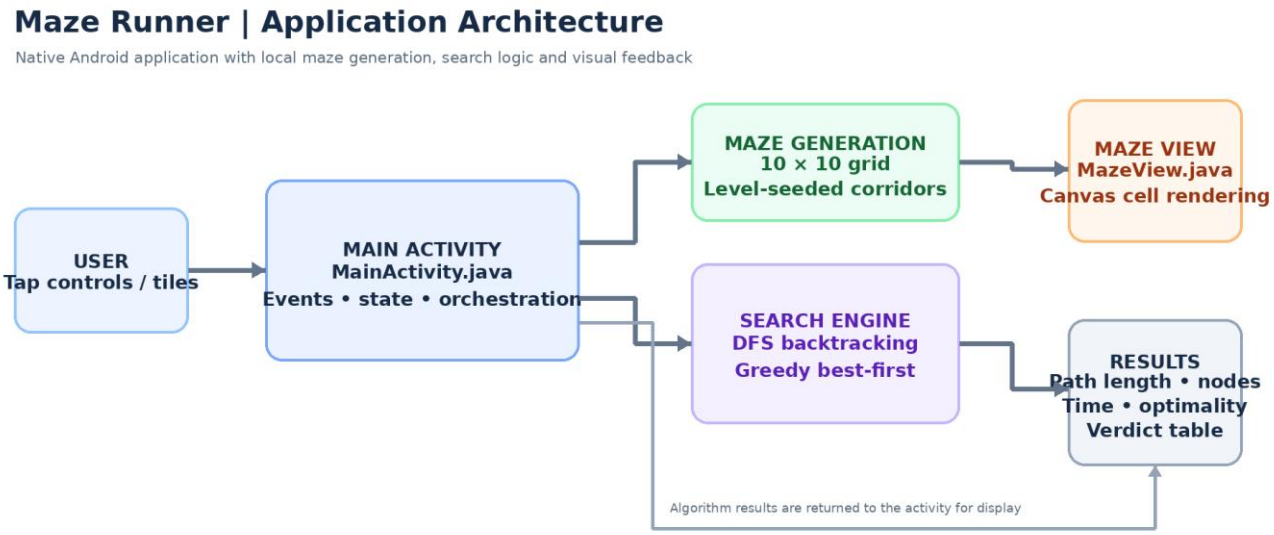


Figure 1. High-level architecture of the supplied Android application.

2.1 Main components

Component	Responsibility
MainActivity.java	Initializes the screen, handles button and grid input, creates mazes, runs both algorithms, stores results and updates the comparison table.
MazeView.java	Extends Android View and draws the 10 × 10 grid, cell colors, start/goal labels, explored states, selected path and player marker using Canvas.
activity_main.xml	Defines the screen layout: title and level badge, status message, maze panel, action buttons, comparison table and result cards.
dialog_story.xml	Provides the introductory dialog shown when the application starts.
Resources	Contains theme colors, button/card backgrounds, launcher assets and application labels.

2.2 Interaction flow

The user chooses a level or resets the maze, then selects PLAY, BRUTE or HEURISTIC. PLAY activates manual movement. BRUTE runs DFS and stores route candidates; HEURISTIC runs the priority-queue search. The activity passes the resulting cell states to MazeView for animation and refreshes the comparison values. When both algorithm results are available, the verdict card compares time, route length and explored nodes.

3. Maze Representation and Generation

3.1 Grid model

The maze is represented by a two-dimensional integer array with 10 rows and 10 columns. A cell value of 1 represents a wall, while 0 represents an open cell. The start is the upper-left cell and the destination is the lower-right cell. Search and player movement are restricted to the four directions stored in the row and column offset arrays: down, right, up and left.

Element	Representation / rule
Grid size	10 × 10 cells (100 possible positions).
Wall	A cell with value 1; cannot be entered.
Open cell	A cell with value 0; may be entered if it is adjacent.
Start	(0, 0), displayed with the label S and a green fill.
Goal	(9, 9), displayed with the label G and a red fill.
Movement	One cell per move, using horizontal or vertical adjacency only.

3.2 Maze generation

Generation begins by setting every cell to a wall. The code then opens a set of guaranteed corridors using selected row positions 1, 3, 5, 7 and 8. For each selected row, it opens a vertical corridor from the start-side column, a horizontal corridor across that row, and a vertical corridor down the goal-side column. The resulting safe cells are opened before additional openings are considered.

Additional openings are introduced only in the interior columns. The opening probability is 0.08 for EASY, 0.18 for MEDIUM and 0.08 for HARD. A Java Random instance is initialized with a level-based seed (level × 100), making the pattern repeatable for a given level. Consequently, resetting the same level recreates its seeded layout; it does not produce a fresh random layout each time. EASY and HARD use the same additional-opening probability but different seeds, so the labels do not represent a strictly increasing probability of open cells.

3.3 Visual encoding

Visual state	Color used in MazeView	Meaning
Wall	#1E293B (dark slate)	Blocked cell.
Open cell	#F1F5F9 (near white)	Walkable cell not currently highlighted.
Start	#22C55E (green)	Initial location.
Goal	#EF4444 (red)	Destination cell.
Algorithm path	#3B82F6 (blue)	Cells in the selected route.
Explored cell	#FBBF24 (amber)	Cell examined by heuristic search.
Player	#F97316 (orange)	Current player position.
Player route	#A855F7 (purple)	Cells already traversed manually.

MazeView draws rounded cells with a small gap between them. It also places S and G labels on the endpoint cells and displays a circular marker for the player. The colors make the current state understandable without requiring the learner to read internal algorithm variables.

4. Brute-Force Search: Depth-First Search

The application labels its exhaustive route search as BRUTE FORCE and implements it with recursive depth-first search (DFS). Starting at (0, 0), the method follows one available neighboring cell at a time. If a branch reaches a wall, a boundary, a cell already present in the current route, or a route that cannot improve the best route already found, the branch is stopped. The method then backtracks and tries another direction.

The search uses a path-local visited array: a cell is marked while it is part of the current recursive route and unmarked when the method returns from that branch. This allows the same cell to be considered on a different candidate route while preventing cycles within one route. When the goal is reached, the current route is copied. The shortest route is retained separately, while at most five discovered routes are kept for Previous/Next browsing in the interface.

Algorithm 1. DFS with backtracking and best-route pruning

```
DFS(cell, currentRoute):
    if cell is outside the grid or is a wall: return
    if cell is already visited on currentRoute: return
    if a bestRoute exists and length(currentRoute) >= length(bestRoute): return

    mark cell visited
    append cell to currentRoute

    if cell is the goal:
        copy currentRoute as a completed route
        update bestRoute if this route is shorter
        retain route for browsing if fewer than five are stored
    else:
        for each orthogonal neighbor:
            DFS(neighbor, currentRoute)

    remove last cell from currentRoute
    unmark cell visited
```

4.1 Why DFS is used as the reference

On the fixed finite grid, the DFS routine explores feasible simple routes and uses the best route found to prune branches that are already too long to improve it. This makes the shortest route found by the routine the reference used by the application when it checks whether the heuristic result is optimal. The path browser is intentionally limited to five stored routes, but that display limit is separate from the best-route variable.

4.2 Complexity and trade-off

The number of simple routes in a graph can grow exponentially with the size and openness of the search space. Therefore, DFS route enumeration has a potentially exponential worst-case time cost, even though the current 10×10 maze keeps the demonstration manageable and the best-length pruning reduces unnecessary branches. Recursive depth and the current path require $O(V)$ auxiliary space for V cells; the application also stores a best route and up to five displayable route copies.

DFS is useful here because it provides a transparent way to explore alternatives and establish a shortest-path reference for the small maze. Its cost is that it may inspect many branches before finishing.

5. Heuristic Search: Greedy Best-First Search

The second strategy is greedy best-first search. It uses an estimate of how far a cell is from the goal and gives priority to cells with the smallest estimate. For a cell at row r and column c, the application calculates Manhattan distance to (9, 9):

$$h(r, c) = |9 - r| + |9 - c|$$

Manhattan distance counts the horizontal and vertical steps needed to reach the destination if there were no obstacles. The algorithm inserts the start node into a priority queue and repeatedly removes the node with the lowest h value. It records explored cells and parent coordinates. Once the goal is reached, the route is reconstructed by following parent links back to the start and reversing that sequence.

Algorithm 2. Greedy best-first search

```
create a priority queue ordered by the smallest h(cell)
insert start (0, 0); mark it closed

while the queue is not empty:
    current = remove cell with the smallest h(current)
    record current as explored

    if current is the goal:
        stop with success

    for each orthogonal neighbor of current:
        if neighbor is inside the grid, open and not closed:
            mark neighbor closed
            store current as neighbor's parent
            insert neighbor into the priority queue

if the goal was found:
    follow parent links from goal to start
    reverse the collected sequence to obtain the route
```

5.1 Why this is not A*

The supplied implementation is greedy best-first search, not A*. Its priority is based only on h(n), the estimated remaining distance. A* normally uses f(n) = g(n) + h(n), where g(n) is the cost already travelled and h(n) is the estimated cost remaining. Because Maze Runner does not include g(n) in the priority score, it may prefer a cell that looks close to the goal even when the route taken so far has been inefficient.

5.2 Complexity and trade-off

For the current grid, each cell is added to the priority queue at most once because it is marked closed when first discovered. With V cells and E adjacency edges, priority-queue processing is approximately O((V + E) log V) time and O(V) storage. The search usually concentrates on cells that appear closer to the goal, but walls can make the Manhattan estimate misleading. Greedy best-first search therefore does not guarantee a shortest route.

6. Android Implementation and User Interface

6.1 Application structure

The project is implemented as a Java-based Android application using a standard Activity and XML layout. MainActivity coordinates the interface and algorithm state. MazeView is a custom Android View that renders the maze through the Canvas API. The supplied Gradle configuration identifies application version 1.0, Java 11 source/target compatibility, minimum Android API level 24 and compile/target SDK level 37.

File / area	Implementation role
MainActivity.java	Maze generation, DFS, greedy best-first search, manual play, animations, route state, timing and verdict text.
MazeView.java	Custom Canvas drawing of the grid and visual overlays.
activity_main.xml	Screen arrangement and comparison table.
dialog_story.xml	Introductory story dialog and start action.
res/values/colors.xml	Theme, surface, accent and maze color definitions.
AndroidManifest.xml	Declares Maze Runner application and MainActivity launcher entry point.

6.2 Main screen and user actions

Control / area	User-visible behavior
Level badge / next-level control	Shows EASY, MEDIUM or HARD and cycles to the next level.
PLAY	Starts manual navigation from the start cell.
BRUTE	Runs the recursive DFS route search and animates a stored route.
HEURISTIC	Runs greedy best-first search, animates explored cells and then its route.
Previous / Next path	Browses stored DFS routes, when more than one is available.
Reset / clear	Regenerates the current seeded maze or clears the player's current route.
Comparison table	Displays path length, explored nodes, time, paths found and optimality.
Verdict / heuristic trace	Summarizes the comparison and shows the first 12 heuristic trace entries.

6.3 Manual play

In PLAY mode, the user taps a cell directly adjacent to the current player position. A tap is rejected if it is outside the grid, is not one orthogonal step away, or corresponds to a wall. A valid move advances the player, records the route and increments the move count. Reaching (9, 9) ends the run. If a DFS result is already available, the player route is checked against its path length and the interface reports whether the user matched the reference or used extra moves.

6.4 Animation and feedback

The app uses a Handler on the main thread to reveal cells in sequence. Heuristic exploration is animated before its final path; a DFS route is replayed cell by cell. Status labels and algorithm badges indicate the current state, while the comparison card is updated after a search completes. Computation itself is performed synchronously before the animation is scheduled.

7. Performance Evaluation and Metrics

The application compares the two algorithms on the same generated maze. The learner runs the search methods and observes the values produced for that session. The uploaded project notes do not provide a fixed benchmark table or a set of recorded numerical outputs, so this report describes the measurements and how to interpret them rather than inventing example results.

Metric	How the application obtains it	What it indicates
Path length	Number of moves in the reconstructed route (number of cells minus one).	Route quality; fewer moves means a shorter route.
Nodes explored	DFS counter for visited cells; heuristic count from its explored-cell list.	Amount of search work represented by the implementation.
Execution time (ms)	Elapsed time measured with System.nanoTime() around the search routine.	Observed computation duration on the current device and run.
Paths found	Number of DFS routes retained for browsing, with a maximum of five.	How many completed DFS routes are available in the path browser; not the total number of routes examined.
Optimal?	Heuristic route length is compared with the DFS best-route length.	Whether the heuristic route matches the DFS reference length.
Player moves	Count of valid adjacent moves made by the user.	Compares manual play with the algorithmic reference when available.

7.1 Comparison procedure

1. Generate or select a level and note that the same level uses a repeatable seeded layout.
2. Run BRUTE and record path length, nodes explored, time and routes retained.
3. Run HEURISTIC on the same maze and record path length, explored cells and time.
4. Compare the heuristic path length with the DFS reference and read the verdict card.
5. Optionally use PLAY and compare the player move count with the DFS reference.

7.2 Interpreting the results

A lower execution time means the method completed faster in that run; it does not prove that the route is shorter. Fewer explored nodes indicate less search activity as counted by the implementation, but node counts and path quality measure different things. The DFS method is used as the shortest-path reference, while the greedy method is expected to be efficient in many cases but may take a detour because it ranks cells using only Manhattan distance.

Timing is measured on the device running the app and can vary with hardware, operating-system scheduling and runtime conditions. The app measures the search routine before the animation is played, so the displayed algorithm time does not represent the duration of the visual animation.

8. Testing and Verification Approach

Testing should confirm that maze constraints, algorithm behavior, user movement and result reporting remain consistent. The supplied repository includes two starter test classes: a basic arithmetic unit-test assertion and an instrumentation test that checks the application package name. These tests do not directly exercise maze generation, DFS, greedy search or manual movement. The cases below are therefore a recommended verification plan, not a claim that every case has already been executed.

Test case	Action	Expected behavior
Maze initialization	Launch the app.	A 10 × 10 grid is shown; start and goal are visible; the level label is set.
Level transition	Select the next level.	The level label changes and a level-seeded maze is generated.
DFS route	Run BRUTE.	A valid path is found for the generated maze; route statistics are populated.
DFS path browsing	Use Previous / Next when available.	Only stored routes can be selected; controls are disabled at the ends.
Greedy route	Run HEURISTIC.	Explored cells are animated and a reconstructed route is shown when the goal is reached.
Greedy optimality check	Run both algorithms on one maze.	The table marks the heuristic route YES if its length equals the DFS reference and NO otherwise.
Blocked move	In PLAY, tap a wall cell.	Movement is rejected and the status message explains that the route is blocked.
Non-adjacent move	Tap a non-neighbor cell in PLAY.	The position does not change; the app requests an adjacent tile.
Manual completion	Navigate valid cells to the goal.	The run ends and the number of moves is available for comparison.

8.1 Existing automated test coverage

- ExampleUnitTest.java contains a starter assertion verifying that 2 + 2 equals 4. It does not test the maze algorithms.
- ExampleInstrumentedTest.java reads the target application context and checks that the package name equals com.example.myapplication. It does not validate UI flows or search output.

For stronger verification, algorithm tests should use fixed maze arrays with known reachable and unreachable cases, compare DFS output with a trusted shortest-path result, verify parent reconstruction for greedy search, and check that every returned path contains only open orthogonally adjacent cells. These tests should be added before using the application as a benchmark or extending the maze size.

9. Limitations

1. Small fixed grid: the application is limited to a 10 × 10 maze. It demonstrates the search concepts but does not show how runtime grows across larger graphs.
2. Greedy search is not optimal: the priority queue uses h(n) only and does not account for the cost already travelled. Obstacles may make the estimate misleading.
3. DFS scalability: route enumeration can grow rapidly as the maze becomes more open. Best-route pruning helps, but it does not change the exponential worst-case behavior.
4. Seeded level layouts: a given level is repeatable rather than newly randomized on every reset. EASY and HARD use the same additional-opening probability in the current implementation.
5. Synchronous computation: both search routines execute from UI event handlers before animation. Larger search spaces could delay screen interaction.
6. Device-dependent timings: measured milliseconds vary across devices and runtime conditions; a single observation should not be treated as a stable performance benchmark.
7. Limited automated tests: the included test classes are starter checks and do not validate path correctness, edge cases or interaction behavior.
8. No persistent records: maze results are held in application state; the supplied project does not include a database, result history or export feature.

10. Future Enhancements

- Add breadth-first search and Dijkstra's algorithm to compare uninformed and cost-based shortest-path strategies.
- Add A* using f(n) = g(n) + h(n), with a consistent movement-cost model, and compare it directly with greedy best-first search.
- Allow configurable maze dimensions and controlled random seeds so learners can repeat a case or generate new layouts.
- Introduce automated tests for path validity, shortest-path correctness, no-solution cases, and UI state transitions.
- Move larger search computations to a background executor or coroutine-based worker to keep the interface responsive.
- Record repeated measurements and show averages or distributions instead of relying on one timing observation.
- Add a result history or export option so students can compare several mazes in one experiment.

11. Learning Outcomes

- Represent a maze as a graph of cells and legal movements.
- Understand recursion, depth-first traversal, visited-state tracking and backtracking.
- Understand how a heuristic estimate and a priority queue guide greedy best-first search.
- Distinguish greedy best-first search from A* search.
- Interpret path length, explored nodes and execution time as separate evaluation measures.
- Recognize why a fast search is not necessarily the one that finds the shortest route.
- Connect algorithm logic with user interface rendering, animation and interactive feedback.

12. Conclusion

Maze Runner provides an interactive demonstration of how different search strategies solve the same path-finding problem. Its brute-force implementation uses recursive DFS and backtracking to explore candidate routes and retain a shortest-path reference for comparison. Its heuristic implementation uses greedy best-first search with Manhattan distance to guide exploration toward the goal. Both approaches are displayed on the same 10 × 10 grid, allowing a learner to observe the path, the explored cells and the measured search time.

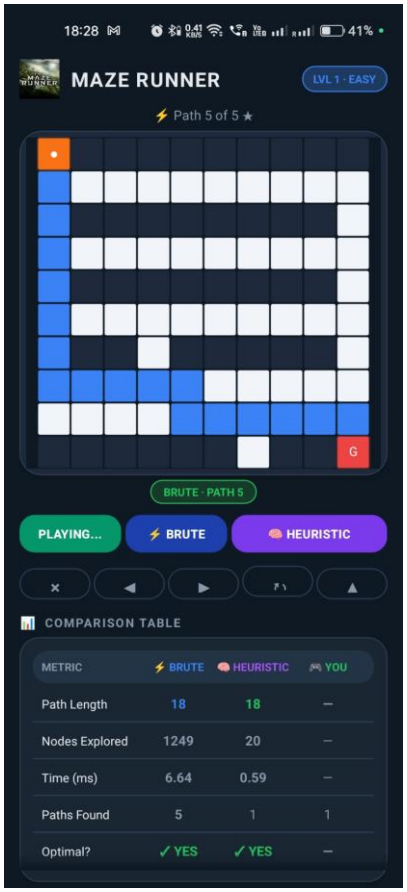
The project highlights an important algorithmic trade-off: exhaustive exploration can provide a reliable shortest-path reference on a small finite maze, but its cost can grow quickly. Greedy best-first search can focus work toward the destination, yet its heuristic ignores the distance already travelled and therefore does not guarantee an optimal route. The manual play mode reinforces this comparison by letting a user test a route and measure the moves taken.

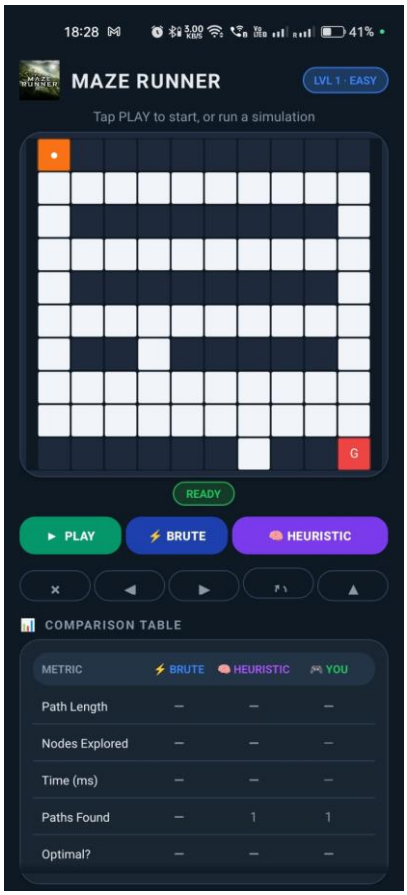
Overall, the application achieves its educational purpose by combining algorithm execution, visualization, quantitative comparison and manual interaction. The next development step should be to add dedicated algorithm tests, more search methods and larger configurable mazes while keeping the comparison reproducible and the interface responsive.

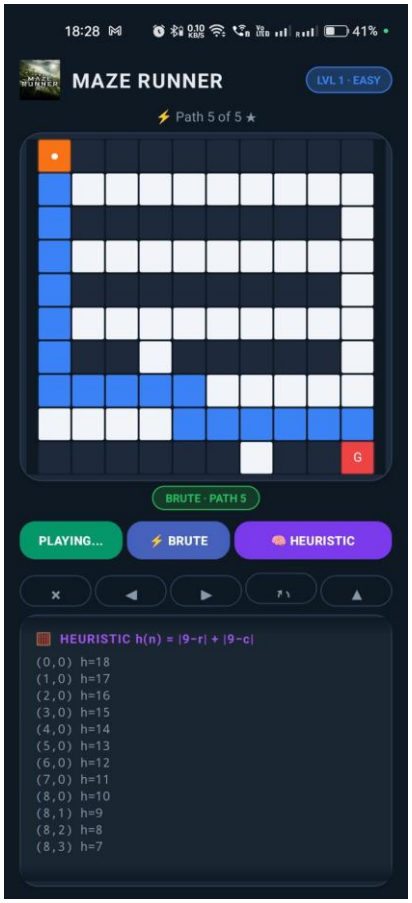
Project source references

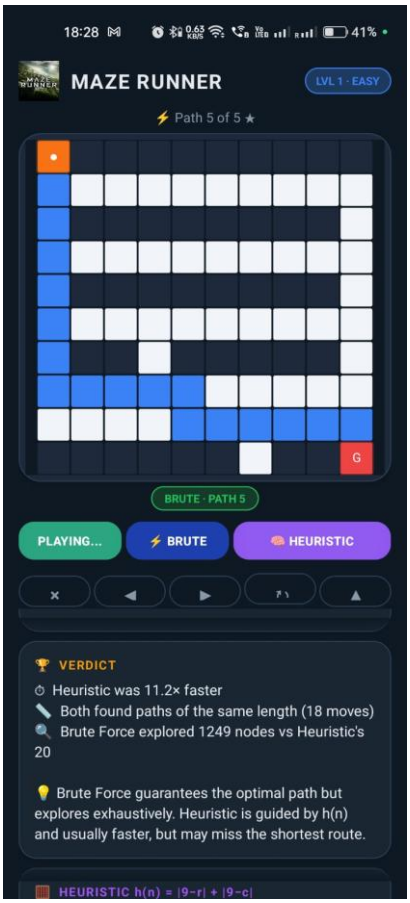
This report is based on the uploaded Maze Runner project and its accompanying report notes. The primary implementation files consulted were MainActivity.java, MazeView.java, activity_main.xml, dialog_story.xml, AndroidManifest.xml, app/build.gradle, colors.xml, ExampleUnitTest.java and ExampleInstrumentedTest.java. The source notes file report.md supplied the project feature summary and the explanation of the two search approaches. The supplied sample PDF was used only as a reference for the visual arrangement of the cover page; its project description and report body were not used as content.

GIT HUB REPO LINK WITH WHOLE SOURCE CODE – https://github.com/Sarthak-Chauhan-Git/MazeRunner


18:28
41%
MAZE RUNNER
LVL 1 · EASY
Path 5 of 5 ★
G
BRUTE · PATH 5
PLAYING...
BRUTE
HEURISTIC
COMPARISON TABLE
METRIC
BRUTE
HEURISTIC
YOU
Path Length
18
18
–
Nodes Explored
1249
20
–
Time (ms)
6.64
0.59
–
Paths Found
5
1
1
Optimal?
✓ YES
✓ YES
–


18:28
41%
MAZE RUNNER
LVL 1 · EASY
Tap PLAY to start, or run a simulation
G
READY
PLAY
BRUTE
HEURISTIC
COMPARISON TABLE
METRIC
BRUTE
HEURISTIC
YOU
Path Length
–
–
–
Nodes Explored
–
–
–
Time (ms)
–
–
–
Paths Found
–
1
1
Optimal?
–
–
–


18:28
41%
MAZE RUNNER
LVL 1 · EASY
Path 5 of 5 ★
G
BRUTE · PATH 5
PLAYING...
BRUTE
HEURISTIC
HEURISTIC h(n) = |9−r| + |9−c|
(0,0) h=18
(1,0) h=17
(2,0) h=16
(3,0) h=15
(4,0) h=14
(5,0) h=13
(6,0) h=12
(7,0) h=11
(8,0) h=10
(8,1) h=9
(8,2) h=8
(8,3) h=7


18:28
41%
MAZE RUNNER
LVL 1 · EASY
Path 5 of 5 ★
G
BRUTE · PATH 5
PLAYING...
BRUTE
HEURISTIC
VERDICT
Heuristic was 11.2× faster
Both found paths of the same length (18 moves)
Brute Force explored 1249 nodes vs Heuristic's 20
Brute Force guarantees the optimal path but explores exhaustively. Heuristic is guided by h(n) and usually faster, but may miss the shortest route.
HEURISTIC h(n) = |9−r| + |9−c|